11

关于 CPU 虚拟化的总结对话

教授：那么，同学，你学到什么了吗？

学生：嗯，教授，这问题好像有点诱导啊。我觉得您就是想让我说“学到了”。

教授：没错。不过这也仍然是个真心的问题。拜托，给教授留点面子，好不好？

学生：好吧，好吧。我觉得我确实学到了几样东西。首先，我稍微了解了操作系统是怎样虚拟化（*virtualize*）CPU 的。要弄懂这件事，我得理解一大堆重要的**机制**（*mechanism*）：陷阱（*trap*）和陷阱处理程序（*trap handler*）、时钟中断（*timer interrupt*），还有在进程（*process*）之间切换时，操作系统和硬件必须怎样小心地保存和恢复状态。

教授：很好，很好！

学生：不过，这些交互看起来还是有点复杂；我怎样才能学得更深入呢？

教授：嗯，这是个好问题。我觉得没有什么能代替亲手去做；光是读这些东西，并不能让你真正体会到其中的门道。去做课程项目吧，我敢打赌，做到最后，你差不多就全明白了。

学生：听起来不错。我还能跟您说点什么？

教授：嗯，在你努力弄懂操作系统基本机制的过程中，有没有体会到一点操作系统的设计哲学？

学生：嗯……我想有吧。操作系统好像相当多疑。它想确保自己始终掌控着机器。它希望程序尽可能高效地运行（这也是**受限直接执行**（*limited direct execution*）背后的全部道理），但一旦遇到出错的或者恶意的进程，它也希望能说一句：“啊哈！别急，朋友。”多疑主宰一切，也确实让操作系统一直掌控着机器。也许正因为如此，我们才把操作系统看作资源管理器（*resource manager*）。

教授：确实如此，听起来你开始把这些东西串起来了！不错。

学生：谢谢。

教授：那么，构建在这些机制之上的策略（*policy*）呢？有什么有意思的收获吗？

学生：那方面当然也有一些收获。也许有点显而易见，不过显而易见也可以是好事。比如把短作业挪到队列（*queue*）前面这个思路。自从有一次我在商店买口香糖，排在我

前面的那个人信用卡怎么都刷不了，我就知道这是个好主意了。我跟您说，他可绝对不是什么短作业。

教授：这么说那个可怜的家伙，好像有点不厚道。还有别的吗？

学生：嗯，还有就是，你可以构建一个聪明的调度程序（*scheduler*），让它同时既像 *SJF* 又像 *RR*，*MLFQ* 真挺巧妙的。要构建一个真正的调度程序，看起来挺难的。

教授：确实很难。所以直到今天，用哪个调度程序仍然有争议；比如看看 *Linux* 里 *CFS*、*BFS* 和 *O(1)* 调度程序之间的论战就知道了。还有，不，我不会把 *BFS* 的全名说出来的。

学生：我也不会让您说的！这些关于策略的论战好像会永远争下去；真的存在正确答案吗？

教授：可能没有。毕竟，就连我们自己定的指标都是互相矛盾的：如果你的调度程序周转时间（*turnaround time*）好，响应时间（*response time*）就差，反过来也一样。正如 *Lampson* 所说，也许目标不是找到最好的方案，而是避免灾难。

学生：这有点让人泄气。

教授：好的工程就是这样。不过它也可以很鼓舞人心！其实就看你从什么角度去看了。我个人认为，务实是件好事，而务实的人明白，并不是所有问题都有干净又简单的解决办法。还有什么让你感兴趣的吗？

学生：我真的很喜欢“钻调度程序的空子（*game the scheduler*）”这个想法；下次我在亚马逊的 *EC2* 服务上跑作业的时候，也许可以研究研究。说不定我能从别的毫无防备的（更重要的是，不懂操作系统的）客户那里偷一些 *CPU* 周期！

教授：看来我可能造出了一个怪物！你知道，我可不想被人叫作弗兰肯斯坦教授。

学生：可这不正是您的目的吗？让我们对某样东西感兴趣，兴趣大到会自己去钻研？点燃火种之类的？

教授：我想是吧。不过我没想到这真管用！

 © 2008–23, ARPACI-DUSSEAU